
Predicting CVSS and CWE from CVE Descriptions with Fine-Tuned Small Language Models

Tara Dixit

Jessie Dong

Summer Li

Stanford University

{taradixit, jessiedong, summerli}@stanford.edu

Abstract

Security teams rank newly disclosed vulnerabilities by their CVSS vector, the severity tier that follows from it, and their weakness class. Until April 2026 the National Vulnerability Database supplied these labels for most CVEs. It now enriches only high-priority ones, and most new vulnerabilities arrive as a text description with none of the fields that triage depends on. This paper measures how well those fields can be recovered from the description. Models are trained on CVEs from 2022 and 2023, taken from a pinned NVD snapshot, and tested on all 36,325 usable CVEs from 2024. The comparison includes TF-IDF classifiers, the 3.8B-parameter Phi-3.5-mini used zero-shot, and the same model fine-tuned with LoRA, each scored two ways: by predicting the CVSS score directly or by computing it from the predicted vector. The fine-tuned model is the strongest system on every field, with 69.9% severity accuracy, 70.9% CWE accuracy, and an exact CVSS vector on 48.8% of test CVEs. It leads TF-IDF logistic regression by +2.8 to +4.7 points, with paired bootstrap intervals that exclude zero, but the margin is small and the two systems fail on different CVEs: one or the other is correct on 75.8% of severities. Computing the score from the vector helps the linear model and makes every output internally consistent, at the cost of over-calling CRITICAL. The most common near miss, for both systems, is a single wrong guess about whether the attacker needs privileges. Code, configuration, and per-CVE predictions accompany the paper, and every number in it is regenerated from the pinned snapshot.

1 Introduction

A disclosed vulnerability gets a CVE identifier and a short description. The description says what is wrong. It does not say how urgent the problem is, and a team with a queue of a few hundred CVEs needs to know that first. For this they use structured *enrichment*: a CVSS v3.1 vector recording how the flaw is exploited and what it affects, the base score (0–10) and severity tier that follow from the vector, and a CWE category for the type of weakness ([FIRST, 2019](#)). Scanners, patch managers, and incident responders sort by these fields.

For most CVEs the enrichment came from the National Vulnerability Database (NVD) at NIST. The NVD fell behind in early 2024, and on 15 April 2026 NIST announced that it

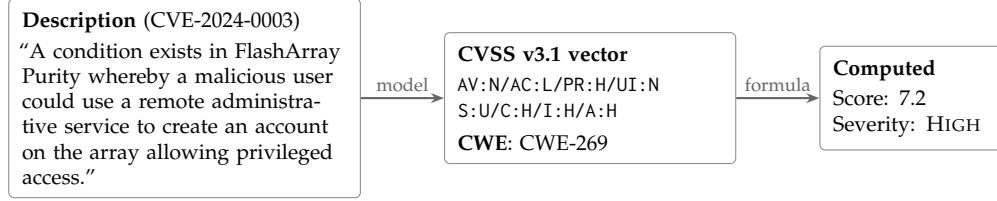


Figure 1: CVE enrichment for CVE-2024-0003. The model reads the description and predicts the eight CVSS components and the CWE. Under formula scoring, the score and severity are then computed from the predicted vector with the CVSS v3.1 equations, so the three always agree.

would from then on enrich only CVEs in CISA’s Known Exploited Vulnerabilities catalog, CVEs in software the federal government uses, and critical software under Executive Order 14028. Everything else is marked “Lowest Priority – not scheduled for immediate enrichment” (National Institute of Standards and Technology, 2026). So for a typical new CVE, a defender now has the description and whatever score the reporting organization chose to attach.

This paper asks how much of the missing enrichment a model can recover from the description alone. The task, called **CVE enrichment** here, is to predict the eight CVSS v3.1 vector components, the base score, the severity tier, and the CWE (Figure 1). Three kinds of system are compared on it: TF-IDF classifiers, a small instruction-tuned language model used zero-shot, and the same model fine-tuned with LoRA.

Two choices shape the evaluation. The first is a split by time. Training uses CVEs from 2022 and 2023 and testing uses every usable CVE from 2024, so no model is scored on a period it has seen. A random split would let near-duplicate descriptions of the same product, written weeks apart, land on both sides, and several public models are trained that way. The second choice is to compute the score rather than predict it. The CVSS base score and severity are fixed functions of the eight components, so a model can predict the components and apply the formula, as Elbaz et al. (2020) and Shahid and Debar (2021) did. This makes score, severity, and vector consistent by construction. The paper measures what the computation adds over predicting the score directly, and whether a language model’s own score agrees with the vector it writes.

On the full test year the fine-tuned model is the strongest system on every field (Figure 2). It reaches 69.9% severity accuracy and 70.9% CWE accuracy, predicts the exact CVSS vector for 48.8% of CVEs, and leads TF-IDF logistic regression by +2.8 to +4.7 points. The margin is modest, and the more useful findings are about where the systems fail. The two systems disagree on a large minority of CVEs, and one or the other is right on 75.8% of severities. When the fine-tuned model misses the vector, it usually misses by one component, and that component is usually privileges required. Both systems do worst on the shortest descriptions and on the LOW tier, which barely exists in the training data. After 2.5 hours of fine-tuning, the model’s written score matches its own vector on 99.8% of CVEs. Used zero-shot, the same model manages 3.4%.

2 Related Work

CVSS vector prediction. Elbaz et al. (2020) predicted each CVSS metric from a bag-of-words description with a linear model and derived the score from the predicted vector. Their emphasis was on decisions that an analyst could inspect at disclosure time. CVSS-BERT (Shahid and Debar, 2021) trained one BERT classifier per CVSS v3.1 metric and computed the score with the equations of the specification. Marchiori et al. (2025) compared

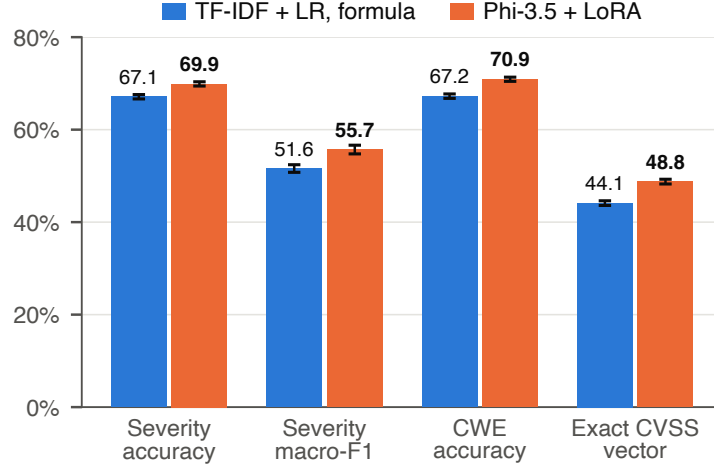


Figure 2: Severity accuracy, severity macro-F1, CWE accuracy, and exact-vector rate for fine-tuned Phi-3.5 and TF-IDF logistic regression with formula scoring on the full 2024 test set (36,325 CVEs). Error bars are 95% bootstrap intervals.

general-purpose LLMs with embedding-based classifiers for the same task and found the classifiers stronger on the impact metrics, the three that ask what an attacker gains in confidentiality, integrity, and availability. The present study keeps the predict-the-vector design and adds three things: a fine-tuned small language model, a strict temporal split, and a direct comparison against score regression.

Severity and CWE classification. VLAI (Bonhomme and Dulaunoy, 2025) fine-tunes RoBERTa on more than 600,000 vulnerabilities to predict the severity tier. Its training set spans many years and includes 2024, so there is no fair way to score it on the test set used here (Section 3). For CWE assignment, Aota et al. (2020) used bag-of-words features with random forests, and Mosievskiy (2026) reports that a 125M-parameter RoBERTa competes with much larger LLMs.

LLMs for vulnerability triage. Talibzade et al. (2025) found that TF-IDF classifiers beat general-purpose LLMs with generic prompts on CWE and severity assignment. They attribute this to the schematic wording of CVE descriptions, and the results below agree. CVE-LLM (Ghosh et al., 2025) fine-tunes an LLM with a domain ontology to evaluate medical-device vulnerabilities, and Torkamani et al. (2025) use LLMs to assign CWEs and severity inside a triage tool. On the related task of predicting exploitation, Roytman (2025) reports that frontier LLMs fall well short of the statistical EPSS model and cost far more to run at scale. Industry systems such as VulnWatch (Databricks, 2025) prioritize vulnerabilities with hosted LLMs. The setting here differs in two ways. Everything runs on local hardware, and every output is scored on vulnerabilities disclosed after the training data.

3 Task and Data

Task. The input is the English description of a CVE. The outputs are the eight CVSS v3.1 base metrics (attack vector, attack complexity, privileges required, user interaction, scope, and the confidentiality, integrity, and availability impacts), the base score, the severity tier (CRITICAL, HIGH, MEDIUM, LOW), and the CWE identifier. Figure 1 shows one example.

Source. NVD records come from the Fraunhofer FKIE mirror of the NVD JSON feeds (Fraunhofer FKIE, 2026), pinned to release v2026.10.05-000016. NVD revises

Table 1: Data splits, with severity columns giving the share of CVEs (%) in each tier.

Split	CVEs	Critical	High	Medium	Low	CWEs
Train (2022–23)	40,436	15.4	39.3	43.7	1.7	464
Validation (2022–23)	10,109	15.4	39.3	43.7	1.7	293
Test (2024)	36,325	13.2	36.2	48.6	1.9	512

records over time, and a fixed release keeps every number in this paper reproducible.

Filtering. A CVE is kept if it has an English description of at least 20 characters, a CVSS v3.x base metric with a severity tier, and a specific CWE. The placeholders NVD-CWE-noinfo and NVD-CWE-Other do not count. When a CVE carries several CVSS entries, the primary score assigned by NIST is used. This leaves 86,870 CVEs from 2022 to 2024. 776 of them use CVSS v3.0 and the rest v3.1, and the base-score equations of the two versions agree on every one.

Temporal split. CVEs with 2022 and 2023 identifiers form a pool of 50,545 CVEs. It is split 80/20 at random, stratified by severity, into 40,436 training and 10,109 validation CVEs. All 36,325 usable CVEs with 2024 identifiers form the test set. Validation data serve only to choose hyperparameters and checkpoints. Table 1 gives the label distribution of each split.

Label distribution. LOW is under 2% of every split. CWEs follow a long tail: the training set has 464 distinct CWEs, the test set 512, and the 100 most frequent training CWEs cover 91.7% of test CVEs.

CVSS calculator. The CVSS v3.1 base-score equations (FIRST, 2019) are implemented in full, including the floating-point-safe round-up that the specification requires. Applied to the true vectors, the calculator reproduces the NVD base score and severity for all 86,870 CVEs. Any error in a formula-based score therefore comes from the predicted vector.

Excluded baseline. VLAI (Bonhomme and Dulaunoy, 2025) is a public severity classifier. Its training data, the CIRCL/vulnerability-scores dataset, uses a random split over all years, and rows sampled from the training split include CVEs from 2024, 2025, and 2026. Scoring it on the 2024 test set would measure memorization as much as generalization, so it is left out.

4 Methods

4.1 Direct and Formula Scoring

Every system predicts the eight vector components and the CWE. The score and severity are then produced in one of two ways. In the *direct* variant, a separate model predicts the score by regression and the severity by classification, or a language model writes both itself. In the *formula* variant, the CVSS v3.1 equations are applied to the predicted components, so score, severity, and vector agree by construction. The TF-IDF systems share one ridge regression for the direct score, so their direct score errors are identical and appear once.

4.2 TF-IDF Baselines

Each description is represented with TF-IDF over unigrams and bigrams, with sublinear term frequency, a minimum document frequency of 2, and at most 50,000 features. Stop words are kept, because “not” and “without” change the meaning of a description. One

Table 2: Validation metrics of the fine-tuned model under formula scoring at each saved checkpoint, on the same 1,000 validation CVEs.

Step	Severity acc.	Macro-F1	CWE acc.	Score MAE	Exact vector
1,000	68.2	50.8	65.4	0.83	52.9
2,000	72.6	56.9	67.2	0.73	55.4
3,000	74.4	58.5	71.1	0.66	62.2
4,000	76.0	62.1	73.4	0.65	63.0
5,000	76.3	58.6	74.2	0.63	63.9

classifier per output is trained with scikit-learn (Pedregosa et al., 2011). Logistic regression (LR) chooses its regularization strength $C \in \{0.3, 1, 3, 10\}$ by validation accuracy, separately for severity, for CWE, and for the components, with one C shared by all eight components. Class-balanced LR is the same with class weights inversely proportional to class frequency. A random forest (RF) with 300 trees for severity and CWE and 100 trees per component is included untuned, with default settings otherwise. The direct score comes from ridge regression with $\alpha \in \{0.3, 1, 3, 10\}$ chosen by validation MAE. The CWE classifiers learn the 100 most frequent training CWEs plus an OTHER class. They are scored against the true CWE, so an OTHER prediction always counts as wrong.

4.3 Phi-3.5-mini

The language model is Phi-3.5-mini-instruct, a 3.8B-parameter decoder (Abdin et al., 2024). Its training data ends in October 2023, before the test period.

Fine-tuning. The model is fine-tuned with LoRA (Hu et al., 2022) in MLX (Hannun et al., 2023) on a single Apple M5 Pro with 64 GB of unified memory. Adapters of rank 16 (scale 20, dropout 0.05) sit on the last 16 transformer layers, giving 12.6M trainable parameters, or 0.329% of the model. Training runs for 5,000 steps at batch size 8, which is one pass over the training set, with a learning rate of 10^{-4} , 100 warm-up steps, and cosine decay to 10^{-5} . The loss covers only the answer tokens. The 561 training examples (1.4%) longer than 768 tokens are dropped to bound memory. Test CVEs are never dropped. The target is one JSON object that lists the eight components first and then the CWE, the score, and the severity, so the model commits to a vector before it writes a score. Checkpoints at steps 1,000 through 5,000 are evaluated on the same 1,000 validation CVEs (Table 2), and the one with the best severity accuracy under formula scoring is kept. Decoding is greedy.

Zero-shot. The same base model, without an adapter, receives the description and a prompt that lists every field and its allowed values (Appendix A). Zero-shot output is verbose and slow to generate for a whole year of CVEs, so it is evaluated on a random sample of 2,000 test CVEs, and every other system is rescored on the same sample.

4.4 Metrics

Severity is evaluated with accuracy and with macro-F1 over the four tiers. Macro-F1 weights LOW as heavily as MEDIUM, which exposes failures on the rare tiers that accuracy hides. CWE prediction uses top-1 accuracy, the score uses mean absolute error (MAE), and the vector uses per-component accuracy, the mean fraction of components correct (*partial match*), and the fraction of CVEs with all eight components correct (*exact match*). A language-model output that is not valid JSON counts as wrong on every field and is left out of the score MAE, with the parse rate reported separately. All 95% confidence intervals come from 1,000 bootstrap resamples of test CVEs.

Table 3: All systems on the full 2024 test set (36,325 CVEs). Values are percentages except score MAE. $\pm$ gives the half-width of the 95% bootstrap interval. Bold marks the best value in each column. The TF-IDF direct systems share one ridge regression, so their score MAE is identical.

System	Severity		CWE	Score	Vector	
	Acc.	Macro-F1	Acc.	MAE $\downarrow$	Partial	Exact
TF-IDF + LR, direct	67.1 $\pm$ 0.5	47.9 $\pm$ 0.6	67.2 $\pm$ 0.5	0.98 $\pm$ 0.01	86.9 $\pm$ 0.2	44.1 $\pm$ 0.5
TF-IDF + LR, formula	67.1 $\pm$ 0.5	51.6 $\pm$ 0.8	67.2 $\pm$ 0.5	0.94 $\pm$ 0.01	86.9 $\pm$ 0.2	44.1 $\pm$ 0.5
TF-IDF + LR balanced, direct	65.7 $\pm$ 0.5	51.9 $\pm$ 0.8	67.2 $\pm$ 0.5	0.98 $\pm$ 0.01	86.0 $\pm$ 0.2	40.5 $\pm$ 0.5
TF-IDF + LR balanced, formula	67.1 $\pm$ 0.5	53.9 $\pm$ 0.9	67.2 $\pm$ 0.5	0.98 $\pm$ 0.01	86.0 $\pm$ 0.2	40.5 $\pm$ 0.5
TF-IDF + RF, direct	66.9 $\pm$ 0.5	47.1 $\pm$ 0.6	65.2 $\pm$ 0.5	0.98 $\pm$ 0.01	86.1 $\pm$ 0.2	41.8 $\pm$ 0.5
TF-IDF + RF, formula	64.0 $\pm$ 0.5	47.8 $\pm$ 0.6	65.2 $\pm$ 0.5	1.03 $\pm$ 0.01	86.1 $\pm$ 0.2	41.8 $\pm$ 0.5
Phi-3.5 + LoRA, direct	69.9 $\pm$ 0.5	55.7 $\pm$ 0.9	70.9 $\pm$ 0.5	0.85 $\pm$ 0.01	87.9 $\pm$ 0.2	48.8 $\pm$ 0.5
Phi-3.5 + LoRA, formula	69.9 $\pm$ 0.5	55.7 $\pm$ 0.9	70.9 $\pm$ 0.5	0.85 $\pm$ 0.01	87.9 $\pm$ 0.2	48.8 $\pm$ 0.5

5 Results

5.1 Main Results

Table 3 reports every system on the 36,325 test CVEs. The fine-tuned Phi-3.5 model leads on every metric: 69.9% severity accuracy, 70.9% CWE accuracy, a score MAE of 0.85, and an exact vector on 48.8% of CVEs. Because every system is scored on the same CVEs, a paired bootstrap gives the difference against TF-IDF logistic regression with formula scoring directly. The fine-tuned model changes severity accuracy by +2.8 points (95% CI +2.4 to +3.1), macro-F1 by +4.1 (+3.0 to +5.2), CWE accuracy by +3.7 (+3.4 to +4.0), score MAE by -0.09 (-0.10 to -0.08), and the exact-vector rate by +4.7 (+4.3 to +5.0). No interval includes zero. The gains are consistent, but they are a few points, and TF-IDF remains a strong baseline for this task.

5.2 Effect of Formula Scoring

For logistic regression, computing the score from the predicted vector lowers score MAE from 0.98 to 0.94 and raises macro-F1 from 47.9% to 51.6%. Accuracy stays at 67.1%. The gain comes from the two rare tiers (Table 4). LOW F1 rises from 2.2% to 10.9%, and CRITICAL F1 from 51.4% to 56.1% as CRITICAL recall climbs from 50.7% to 73.3%.

The improvement has a cost. The formula system predicts CRITICAL for 7,755 CVEs where NVD has 4,800, and its precision on that tier drops from 52.1% to 45.4%. It scores 17.7% of test CVEs at exactly 9.8, the score of the most common critical vector (network, low complexity, no privileges, no interaction, high impact on all three dimensions), against 10.5% in NVD. Its mean signed score error is +0.33, compared with +0.08 for ridge regression. For a triage queue, catching more critical CVEs at the price of more false alarms may be the right trade, but it is a trade.

For the random forest, formula scoring hurts. Accuracy falls from 66.9% to 64.0% and MAE rises from 0.98 to 1.03. Its component predictions are weaker, and the formula passes their errors straight through to the score.

The fine-tuned model shows almost no difference between the two modes. The score it writes matches the CVSS formula for its own vector on 99.8% of test CVEs, and the severity it writes matches its own score on 100.0% (Table 6). Trained on targets that list the components first, the model behaves as if it applies the formula to its own vector.

Table 4: Recall and F1 (%) by true severity tier on the 2024 test set.

System	Critical		High		Medium		Low	
	Rec.	F1	Rec.	F1	Rec.	F1	Rec.	F1
TF-IDF + LR, direct	50.7	51.4	62.2	61.1	77.9	77.1	1.1	2.2
TF-IDF + LR, formula	73.3	56.1	62.6	62.6	71.3	76.8	6.2	10.9
TF-IDF + LR bal., formula	57.1	52.8	61.9	62.2	75.6	77.2	19.2	23.2
Phi-3.5 + LoRA, formula	76.9	59.7	63.6	64.6	75.1	79.0	11.4	19.6

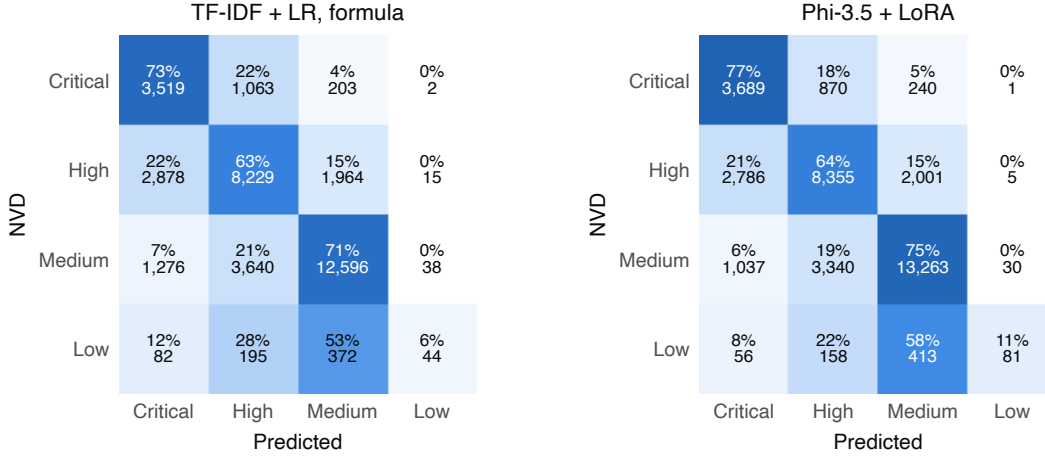


Figure 3: Severity confusion matrices on the 2024 test set, normalized by true tier, for TF-IDF logistic regression with formula scoring (left) and fine-tuned Phi-3.5 (right).

5.3 Error Analysis

Severity tiers. LOW is the hardest tier for every system (Table 4). It makes up under 2% of the training data, and both systems most often label a true LOW CVE as MEDIUM. Class-balanced weights raise LOW recall to 19.2% for TF-IDF. The fine-tuned model reaches 11.4% with no rebalancing at all. Figure 3 shows that most of the remaining confusion is between adjacent tiers.

CVSS components. Figure 4 gives accuracy per component. Attack complexity, scope, attack vector, and user interaction are above 90% for both systems, because descriptions usually state them outright. Privileges required is the hardest component, at 76.5% for TF-IDF and 78.8% for the fine-tuned model, followed by the three impact components. The fine-tuned model gains most on confidentiality (82.9% to 84.8%), integrity (82.9% to 84.5%), and privileges required, the components that depend least on surface keywords. [Marchiori et al. \(2025\)](#) found that general-purpose LLMs trail embedding-based classifiers on the impact components. After fine-tuning, those are among the components where the model gains most.

Rare CWEs. The TF-IDF classifiers can only output the 100 most frequent training CWEs, so they score 0% on the 3,023 test CVEs whose CWE is rarer. The fine-tuned model writes the CWE as text and gets 7.0% of them right (Figure 5). Common, explicitly named weaknesses such as cross-site scripting and SQL injection are close to solved by both systems.

Score distribution. Ridge regression produces a smooth spread of scores that NVD never assigns, and it rarely predicts the extremes (Figure 6). The fine-tuned model’s scores sit on the same discrete values as NVD’s, but it predicts the most common critical score, 9.8, for

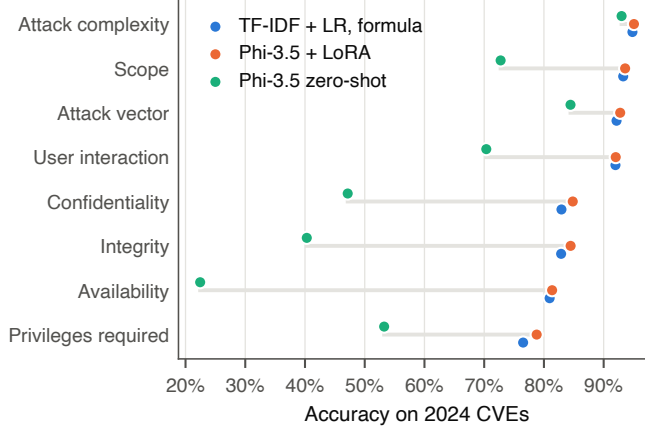


Figure 4: Accuracy per CVSS component on 2024 CVEs. TF-IDF and the fine-tuned model are scored on the full test set, the zero-shot model on its 2,000-CVE sample.

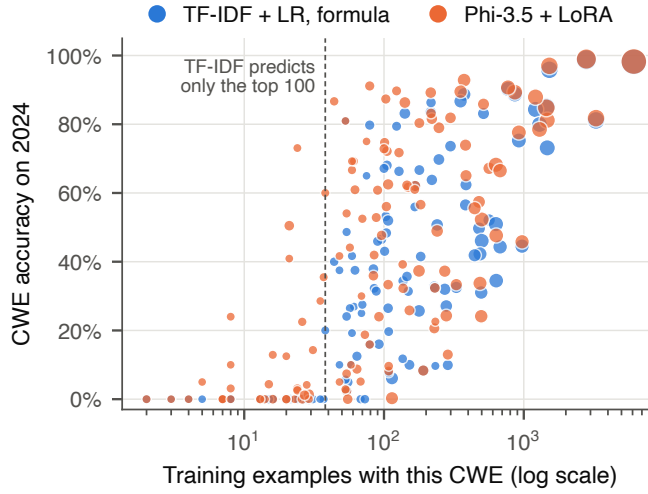


Figure 5: CWE accuracy on the 2024 test set against the number of training examples with each CWE, for every CWE with at least 20 test CVEs. Point size grows with test count.

Table 5: Comparison on the same random sample of 2,000 test CVEs, on which every system is scored.

System	Severity		CWE Acc.	Score MAE ↓	Vector	
	Acc.	Macro-F1			Partial	Exact
TF-IDF + LR, formula	65.3 ±2.1	47.6 ±1.8	66.2 ±2.2	0.99 ±0.06	86.5 ±0.7	42.2 ±2.2
Phi-3.5 zero-shot, direct	20.8 ±1.7	15.4 ±2.4	25.7 ±2.0	1.58 ±0.05	60.5 ±0.7	2.4 ±0.6
Phi-3.5 zero-shot, formula	43.5 ±2.3	26.8 ±1.7	25.7 ±2.0	2.08 ±0.08	60.5 ±0.7	2.4 ±0.6
Phi-3.5 + LoRA, formula	69.6 ±2.0	52.8 ±3.3	68.8 ±2.1	0.87 ±0.05	87.5 ±0.7	48.0 ±2.1

20.0% of test CVEs where NVD assigns it to 10.5%. It predicts CRITICAL for 7,568 test CVEs against 4,800 in NVD, with 48.7% precision, and like formula-scored logistic regression it runs high on average, with a mean signed error of +0.33.

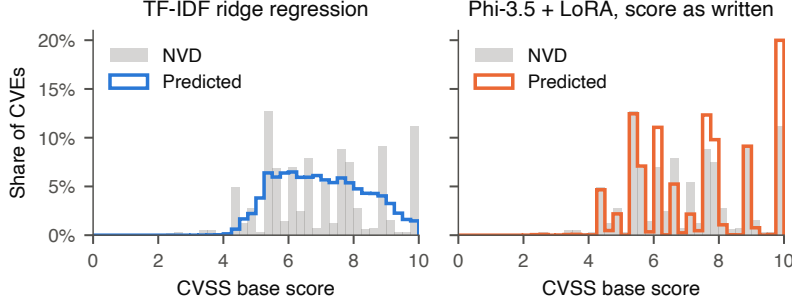


Figure 6: Distribution of predicted and NVD base scores on the 2024 test set in bins of 0.25, for ridge regression (left) and the fine-tuned model with scores as written (right).

5.4 Zero-Shot Baseline

Without fine-tuning, the same base model is far weaker (Table 5). It returned valid JSON for 99.7% of the 2,000 sampled CVEs, and 11 of the parsed outputs used a component value outside the allowed set. The severity it writes is biased upward. It labels 62.4% of the sample CRITICAL, against 13.3% in NVD, and predicts MEDIUM only 9 times although 47.5% of these CVEs are MEDIUM. Its written severity is correct on 20.8% of CVEs. The outputs also contradict themselves. The written score matches the formula for the written vector on 3.4% of CVEs, and the written severity matches the tier of the written score on 37.0% (Table 6). Computing severity from the vector removes the inconsistency and lifts severity accuracy to 43.5%, still far below the fine-tuned model (69.6%) and TF-IDF (65.3%) on the same CVEs. Score MAE gets worse, from 1.58 to 2.08, because the zero-shot vectors are wrong in ways that move the score a long way. The per-component picture explains this (Figure 4). Attack complexity (93%) and attack vector (84%) are close to the trained systems, but availability impact is right on only 22% of CVEs, integrity on 40%, and confidentiality on 47%. The zero-shot model reads how an attack happens reasonably well and guesses what it costs.

Table 6: Self-consistency of language-model outputs (%), where $Score = formula$ is the share of outputs whose written score equals the CVSS v3.1 score of the written vector and $Severity = tier\ of\ score$ is the share whose written severity matches the tier of the written score.

Model	Valid JSON	Score = formula	Severity = tier of score
Phi-3.5 zero-shot	99.7	3.4	37.0
Phi-3.5 + LoRA	100.0	99.8	100.0

5.5 Training and Inference Cost

Validation severity accuracy rose at every checkpoint, and the last one (step 5,000) was selected (Table 2, Figure 7). Macro-F1 peaked earlier, at step 4,000 with 62.1%, so a user who cares most about the rare tiers might prefer that checkpoint. Table 7 gives wall-clock times. Fine-tuning took 2.5 hours and scoring the full test year 4.9 hours, against 9 minutes for all TF-IDF baselines together, including tuning.

Table 7: Wall-clock time on a single Apple M5 Pro (64 GB), with inference run in small batches of at most 32 CVEs and without optimization for speed.

Step	Time
All TF-IDF baselines, tuning and test (CPU)	9 min
LoRA fine-tuning, 5,000 steps	2.5 h
LoRA inference, 36,325 test CVEs	4.9 h

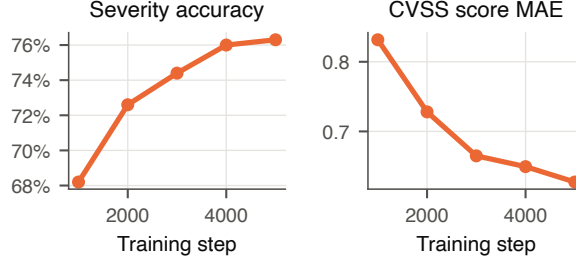


Figure 7: Validation severity accuracy and score MAE of the fine-tuned model with formula scoring at each saved checkpoint, on 1,000 validation CVEs.

6 Discussion

6.1 The Two Systems Fail on Different CVEs

A difference of a few points in accuracy says little about whether two systems make the same mistakes. On this test set they do not. Both systems get the severity right on 61.3% of CVEs and both get it wrong on 24.2%. On the remaining 8.6% only the fine-tuned model is right, and on 5.9% only TF-IDF is. One or the other is therefore correct on 75.8% of CVEs, well above the 69.9% of the better system alone. The pattern repeats for the CWE (either correct on 73.8%, against 70.9%) and for the exact vector (53.3% against 48.8%). No ensemble was tested here, so this is an upper bound rather than a result, but it suggests that the two systems read descriptions differently and that a combination would be worth trying.

6.2 What a Near Miss Looks Like

The exact-vector rate is a severe metric: one wrong component out of eight counts as a miss. Most misses are small. The fine-tuned model gets every component right on 48.8% of test CVEs, exactly one wrong on 25.5%, two wrong on 12.7%, and three or more wrong on 13.0%. TF-IDF has the same shape (44.1%, 28.1%, 14.1%, and 13.6%). When exactly one component is wrong, it is privileges required 38% of the time for the fine-tuned model and 38% for TF-IDF. Availability impact is a distant second, at 17% and 16%.

Table 8 shows what this looks like on individual CVEs. CVE-2024-0003, the running example, is a privileges-required miss. The description says a “malicious user” can create an account through “a remote administrative service.” NIST read “administrative” as meaning the attacker already holds high privileges and scored the CVE 7.2. The model read “malicious user” as an outsider, set privileges required to none, and produced 9.8. Every other component matched. The same single-word ambiguity turns a HIGH into a CRITICAL, which is why this one component accounts for so much of the gap between partial-match and exact-match accuracy. CVE-2024-0008 is a user-interaction miss of the same kind. PAN-OS management sessions that do not expire are scored 8.8 by NVD, with user interaction required, and the model, seeing no mention of a user, scored them 9.8. In both

Table 8: Test CVEs discussed in Section 6, with the NVD labels and the output of the fine-tuned model. The last column lists the vector components the model got wrong.

CVE	Description	NVD	Predicted	Wrong
CVE-2024-0003	A condition exists in FlashArray Purity whereby a malicious user could use a remote administrative service to create an account on the array allowing privileged access.	7.2 High, CWE-269	9.8 Critical, CWE-287	PR
CVE-2024-0008	Web sessions in the management interface in Palo Alto Networks PAN-OS software do not expire in certain situations, making it susceptible to unauthorized access.	8.8 High, CWE-613	9.8 Critical, CWE-613	UI
CVE-2024-0257	RoboDK v5.5.4 is vulnerable to heap-based buffer overflow while processing a specific project file. The resulting memory corruption may crash the application.	3.3 Low, CWE-122	7.8 High, CWE-787	C, I, A
CVE-2024-10214	Mattermost versions 9.11.X <= 9.11.1, 9.5.x <= 9.5.9 incorrectly issues two sessions when using desktop SSO - one in the browser and one in desktop with incorrect settings.	3.5 Low, CWE-303	9.8 Critical, CWE-287	PR, UI, C, I, A
CVE-2024-0204	Authentication bypass in Fortra’s GoAnywhere MFT prior to 7.4.1 allows an unauthorized user to create an admin user via the administration portal.	9.8 Critical, CWE-425	9.8 Critical, CWE-287	none
CVE-2024-0149	NVIDIA GPU Display Driver for Linux contains a vulnerability which could allow an attacker unauthorized access to files. A successful exploit of this vulnerability might lead to limited information disclosure.	3.3 Low, CWE-125	3.3 Low, CWE-200	none

Table 9: Mean absolute score error by true severity tier on the 2024 test set.

System	Critical	High	Medium	Low
TF-IDF ridge regression (direct)	1.23	0.80	0.96	2.91
TF-IDF + LR, formula	0.59	0.81	1.04	3.26
Phi-3.5 + LoRA, formula	0.53	0.76	0.92	2.90

cases the model’s reading is defensible from the text. The information it lacks is not in the description.

6.3 The Low Tier

Every system is worst on LOW, and the reason is not only that the tier is rare. Table 9 gives score MAE by true tier. On CRITICAL CVEs the fine-tuned model’s score is off by 0.53 on average, on HIGH by 0.76, on MEDIUM by 0.92, and on LOW by 2.90. Ridge regression, which never commits to a vector, does no better on them, at 2.91. A LOW CVE that the model gets wrong is usually not wrong by one tier.

The examples show why. CVE-2024-0257 describes a heap-based buffer overflow in RoboDK that “may crash the application.” NVD scored it 3.3, with limited impact, because the attack needs a crafted project file and the worst outcome mentioned is a crash. The model scored it 7.8, with high impact on all three dimensions, which is the typical score for a heap overflow in the training data. It applied the prior for the weakness type and ignored the limiting clause. CVE-2024-10214, a Mattermost bug that issues two sessions during desktop single sign-on, is scored 3.5 by NVD and 9.8 by the model, which read a session-handling flaw as an authentication bypass. Both errors run in the same direction.

Table 10: Severity accuracy and exact-vector rate (%) by quartile of description length on the 2024 test set. Quartile boundaries are 194, 285, and 444 characters.

System	Shortest quartile		Second		Third		Longest quartile	
	Sev.	Vector	Sev.	Vector	Sev.	Vector	Sev.	Vector
TF-IDF + LR, formula	59.7	38.8	67.2	38.4	71.0	50.7	70.6	48.7
Phi-3.5 + LoRA, formula	61.7	42.0	69.7	42.0	74.4	55.7	73.9	55.5

When a description contains words associated with serious weaknesses, the model scores the weakness rather than the sentence. Class-balanced weights help TF-IDF on this tier, and the equivalent for the fine-tuned model, oversampling LOW during training, was not tried.

6.4 Short Descriptions

Table 10 splits the test set into quartiles of description length. Both systems do worst on the shortest quarter, under 194 characters, where the fine-tuned model reaches 61.7% severity accuracy and TF-IDF 59.7%. Accuracy rises through the middle quartiles and levels off, with the fine-tuned model at 74.4% on the third quartile and 73.9% on the longest. The exact-vector rate follows the same curve. The shortest descriptions are often a product name and a weakness class with nothing else, as in CVE-2024-0057 (".NET, .NET Framework, and Visual Studio Security Feature Bypass Vulnerability"), and for those the score has to come from the prior for that vendor and weakness. The fine-tuned model’s lead over TF-IDF is similar in every quartile, so the extra capacity does not specifically rescue short inputs.

6.5 CWE Disagreements

Two of the examples in Table 8 have an exact vector and a wrong CWE. For CVE-2024-0204, an authentication bypass in GoAnywhere MFT, NVD assigned CWE-425 (direct request) and the model CWE-287 (improper authentication). For CVE-2024-0149, an NVIDIA driver flaw that allows “unauthorized access to files” leading to “limited information disclosure,” NVD assigned CWE-125 (out-of-bounds read) and the model CWE-200 (exposure of sensitive information). In the second case the description never mentions a bounds violation. The NVD label records what the analyst learned from the advisory, and the model’s label records what the description says. Some share of the CWE errors reported in Table 3 are of this kind, and the CWE accuracy of any description-only system is bounded by it.

6.6 Choosing a System

The results point to a simple division. TF-IDF logistic regression trains in minutes, needs no GPU, and gives up a few points on every field. Formula scoring should be used with it, since it costs nothing and fixes the rare tiers, unless precision on CRITICAL matters more than recall. The fine-tuned model is better on every field, produces consistent scores and vectors, and can name CWEs outside the frequent set. It costs 2.5 hours to train and about half a second per CVE at inference, which is fast enough for the daily volume of new CVEs. The zero-shot model should not be used for this task. Its written severities are wrong more often than they are right, and even the vector-derived ones trail both trained systems. Whichever system is chosen, the LOW tier and the privileges-required component should be treated as unreliable, and a CVE scored 9.8 by either trained system deserves a second look before it jumps the queue.

7 Limitations

Label noise. NVD vectors are treated as ground truth, but analysts disagree, NVD revises records, and reporting organizations sometimes score the same CVE differently. Section 6 shows several errors that a reviewer would accept as alternative readings. The data snapshot is pinned so that the reported numbers stay fixed as the labels change.

Single split and seed. Evaluation covers one test year, and each model is trained once. The bootstrap intervals capture the sampling of test CVEs and nothing about variation from training. Only one 3.8B model was fine-tuned, for one epoch, and its validation accuracy was still rising at the end (Table 2). Larger models or longer training may do better. Large hosted models were not run, and the fine-tuned model was trained without any rebalancing of the rare tier.

No ensemble. The disagreement between the two trained systems (Section 6) is reported as an upper bound on what a combination could achieve. No combination was built or tested.

CVSS version. New CVEs increasingly carry CVSS v4.0 vectors. The formula approach carries over, since v4.0 also computes the score from the vector, but nothing here was tested on v4.0 data.

Zero-shot sample. The zero-shot model is evaluated on 2,000 random test CVEs rather than the full year, and the intervals in Table 5 reflect the smaller sample.

Input length. Only English descriptions are used, and descriptions are cut at 2,000 characters, which affects 1.4% of test CVEs.

8 Conclusion

A 3.8B-parameter model fine-tuned for 2.5 hours predicts every NVD enrichment field more accurately than strong TF-IDF baselines on a full year of later CVEs. The margins are a few points. TF-IDF remains a fast baseline that any new method should have to beat, and formula scoring, which costs nothing, is the single change that helps it most. The two trained systems fail on different CVEs, and either one is right on 75.8% of severities, which is the most promising lead in these results. Where the fine-tuned model misses, it misses in ways that follow from the text: a single ambiguous word about privileges, a weakness class whose typical score overrides a limiting clause, or a description too short to carry the information at all. LOW remains poorly predicted by every system.

Several extensions follow directly from these findings. Oversampling LOW or weighting its loss should be tried for the fine-tuned model, since the same change helped TF-IDF. A combination of the two trained systems, by agreement or by confidence, would test whether the 75.8% upper bound can be approached. A second training epoch is cheap and the validation curve suggests it would help. Larger open models fine-tuned the same way, and an evaluation on CVSS v4.0 vectors, would show whether the results hold beyond the setting studied here.

References

- Marah Abdin et al. Phi-3 technical report: A highly capable language model locally on your phone, 2024.
- Masaki Aota, Hideaki Kanehara, Masaki Kubo, Noboru Murata, Bo Sun, and Takeshi Taka-

-
- hashi. Automation of vulnerability classification from its description using machine learning. In *IEEE Symposium on Computers and Communications (ISCC)*, 2020.
- Cédric Bonhomme and Alexandre Dulaunoy. VLAI: A RoBERTa-based model for automated vulnerability severity classification, 2025.
- Databricks. VulnWatch: AI-enhanced prioritization of vulnerabilities. <https://www.databricks.com/blog/vulnwatch-ai-enhanced-prioritization-vulnerabilities>, 2025.
- Clément Elbaz, Louis Rilling, and Christine Morin. Fighting n-day vulnerabilities with automated CVSS vector prediction at disclosure. In *Proceedings of the 15th International Conference on Availability, Reliability and Security (ARES)*, 2020. doi: 10.1145/3407023.3407038.
- FIRST. Common vulnerability scoring system v3.1: Specification document. <https://www.first.org/cvss/v3.1/specification-document>, 2019.
- Fraunhofer FKIE. NVD JSON data feeds, release v2026.10.05-000016. <https://github.com/fkie-cad/nvd-json-data-feeds>, 2026.
- Rikhiya Ghosh, Hans-Martin von Stockhausen, Martin Schmitt, George Marica Vasile, Sanjeev Kumar Karn, and Oladimeji Farri. CVE-LLM: Ontology-assisted automatic vulnerability evaluation using large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 2025. doi: 10.1609/aaai.v39i28.35139.
- Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on apple silicon. <https://github.com/ml-explore/mlx>, 2023.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Francesco Marchiori, Denis Donadel, and Mauro Conti. Can LLMs classify CVEs? investigating LLMs capabilities in computing CVSS vectors. In *TrustAI CyberSec Workshop*, 2025.
- Nikita Mosievskiy. Fine-tuning RoBERTa for CVE-to-CWE classification: A 125M parameter model competitive with LLMs, 2026.
- National Institute of Standards and Technology. NIST updates NVD operations to address record CVE growth. <https://www.nist.gov/news-events/news/2026/04/nist-updates-nvd-operations-address-record-cve-growth>, April 2026. Accessed 2026-10-05.
- Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Michael Roytman. Benchmarking LLMs on the vulnerability prioritization task. *Empirical Security*. <https://research.empiricalsecurity.com/research/benchmarking-llms>, 2025.
- Mustafizur R. Shahid and Hervé Debar. CVSS-BERT: Explainable natural language processing to determine the severity of a computer security vulnerability from its description. In *20th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2021.
- Rustam Talibzade, Idilio Drago, and Francesco Bergadano. On using LLMs for vulnerability classification. In *Proceedings of the 2025 Workshop on Large AI Systems and Models with Privacy and Security Analysis (LAMPS)*, 2025. doi: 10.1145/3733800.3763267.
-

Mohammad Jalili Torkamani, Joey NG, Nikita Mehrotra, Mahinthan Chandramohan, Padmanabhan Krishnan, and Rahul Purandare. Streamlining security vulnerability triage with large language models, 2025.

A Prompts

Both prompts are sent as a single user turn through the Phi-3.5 chat template, and descriptions longer than 2,000 characters (1.4% of test CVEs) are cut at that length.

Fine-tuned model (user turn).

You are a cybersecurity expert. Given the CVE description below, predict the NVD enrichment fields.
Return ONLY a JSON object with no extra text.

CVE Description:
<CVE description>

Zero-shot model (user turn).

You are a cybersecurity expert specializing in vulnerability analysis. Given a CVE description, predict the NVD CVSS v3.1 enrichment fields.

CVE Description:
<CVE description>

Respond ONLY with one JSON object with exactly these keys and allowed values:
{
 "attack_vector": "NETWORK"|"ADJACENT_NETWORK"|"LOCAL"|"PHYSICAL",
 "attack_complexity": "LOW"|"HIGH",
 "privileges_required": "NONE"|"LOW"|"HIGH",
 "user_interaction": "NONE"|"REQUIRED",
 "scope": "UNCHANGED"|"CHANGED",
 "confidentiality": "NONE"|"LOW"|"HIGH",
 "integrity": "NONE"|"LOW"|"HIGH",
 "availability": "NONE"|"LOW"|"HIGH",
 "cwe": "CWE-<number>",
 "cvss_score": "<number 0.0-10.0>",
 "severity": "CRITICAL"|"HIGH"|"MEDIUM"|"LOW"}
}

B Reproducibility

All code, configuration files, and per-CVE predictions are in the project repository, the NVD snapshot is pinned to FKIE release v2026.10.05-000016, and each step below is a single command that regenerates its part of the paper.

- Data preparation with `download_data.py` and `prepare_splits.py`.
- TF-IDF baselines with `baselines_classical.py`.
- LoRA training with `mlx_lm.lora` and `scripts/lora_phi35.yaml`.
- Checkpoint selection with `select_checkpoint.py`.
- Evaluation with `evaluate_lora.py`.
- Tables, figures, and numbers in the text with `make_tables.py` and `plot_results.py`.